Gleichungssysteme und Matrixzerlegungen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- **Algorithmen**: Komplexität, Algorithmustypen (direkte Methoden)
- **Fehleranalyse**: Stabilität, Kondition von Problemen
- Matrixnormen, Orthogonalmatrizen
- **Lineare Algebra I**
  - Lösung linearer Gleichungssysteme (grundlegende Methoden)
  - Dreiecksmatrizen, Symmetrische Matrizen, Positiv definite Matrizen
  - Matrixinverse (wir lernen, dass wir sie *NICHT* berechnen...!)

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\boldsymbol{U} \in \mathbb{R}^{n \times n}$ eine obere Dreiecksmatrix mit $u_{ii} \neq 0$. Mit welchem Aufwand lässt sich $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{c}$ lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. Gar nicht — dazu muss man erst $\boldsymbol{U}^{-1}$ berechnen.

Frage 2: Was bedeutet "$\boldsymbol{A} \in \mathbb{R}^{n \times n}$ ist *positiv definit*"?

1. Alle Einträge von $\boldsymbol{A}$ sind positiv.
2. $\det(\boldsymbol{A}) > 0$.
3. $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} > 0$ für alle $\boldsymbol{x} \neq \boldsymbol{0}$.
4. Alle Eigenwerte von $\boldsymbol{A}$ sind verschieden.

Numerische lineare Algebra

- Wir kümmern uns nun um numerische Methoden für die lineare Algebra:
  - Matrizenrechnung
  - Lineare Gleichungssysteme
  - Kleinste-Quadrate-Probleme
  - Eigenwerte
  - ...
- Dabei achten wir insbesondere auf `Kondition` und `Komplexität`.

Matrizenrechnung

Matrizenrechnung

- Die algorithmische Berechnung folgt für gewöhnlich den direkten mathematischen Formeln

$$\boldsymbol{A} + \boldsymbol{B} = \begin{pmatrix} a_{11} + b_{11} & \cdots & a_{1m} + b_{1m} \\ \vdots & & \vdots \\ a_{n1} + b_{n1} & \cdots & a_{nm} + b_{nm} \end{pmatrix},$$

$$\boldsymbol{AB} = \begin{pmatrix} \sum_{i=1}^{m} a_{1i} b_{i1} & \cdots & \sum_{i=1}^{m} a_{1i} b_{ik} \\ \vdots & & \vdots \\ \sum_{i=1}^{m} a_{ni} b_{i1} & \cdots & \sum_{i=1}^{m} a_{ni} b_{ik} \end{pmatrix}.$$

Quiz

Was ist die algorithmische Komplexität für die Addition von $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{n \times m}$?

1. $O(nm)$
2. $O(n + m)$
3. $O(n^2 m^2)$

Quiz

Was ist die algorithmische Komplexität für die Multiplikation von $\boldsymbol{A} \in \mathbb{R}^{n \times m}$, $\boldsymbol{B} \in \mathbb{R}^{m \times k}$?

1. $O(nk)$
2. $O(nmk)$
3. $O(nm + k)$

Matrixmultiplikation

- Für die Multiplikation von zwei $(n \times n)$ Matrizen gibt es Algorithmen mit besserer Komplexität.
- Für $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{n \times n}$:
  - Strassen (1969): $\approx O(n^{2.8})$
  - Alman et al. (2024): $\approx O(n^{2.372})$ (bester bekannter Exponent)



Quelle: Titelbild *Nature* **610** (6. Oktober 2022) zu AlphaTensor (Fawzi et al., 2022).

- Die Konstante, die hinter $O(\cdot)$ versteckt wird, ist aber meist so groß, dass die Algorithmen kaum verwendet werden.
- Wir rechnen der Einfachheit halber immer mit $O(nmk)$.

Lineare Gleichungssysteme

Lineare Gleichungssysteme

- Wir beschäftigen uns nun mit der Lösung eines LGS:

$$\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$$

- Das ist nur sinnvoll, wenn es (genau) eine Lösung gibt. Also nehmen wir an, dass $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ invertierbar ist.
- Die Lösung kann als $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$ geschrieben werden.
- Das numerische Invertieren einer Matrix ist aber
  - relativ instabil und
  - oft langsamer als direktes Lösen des LGS.

Deshalb:

Invertiere niemals eine Matrix!

Demo:

- LGS mit schlecht konditionierter Matrix mit $\kappa_2(\boldsymbol{A}) \approx 10^{14}$
- wahre Lösung $\boldsymbol{x} = (1, \ldots, 1)^\top$:

```
n <- 11 # Hilbert-Matrix:
A <- outer(1:n, 1:n,
           function(i, j) 1 / (i + j - 1))
x <- rep(1, n)
b <- A %*% x

range((solve(A) %*% b) - x) # Fehler über Inverse
range(solve(A, b) - x)      # Fehler direkte Lösung
```

```
[1] -0.195  0.288
[1] -0.000260  0.000332
```

$\Longrightarrow$ maximaler Fehler über $\boldsymbol{A}^{-1}$: ≈ 0.3;
bei direktem Lösen: $\approx 3 \cdot 10^{-4}$
— ca. **1000× genauer** (und schneller).

Lineare Gleichungssysteme

Wir erinnern uns an das Gauss'sche Eliminationsverfahren:

1. Bringe $(\boldsymbol{A} \mid \boldsymbol{b})$ in Zeilenstufenform $(\boldsymbol{U} \mid \boldsymbol{c})$ mit oberer Dreiecksmatrix $\boldsymbol{U}$.
2. Löse das System

$$\begin{array}{ccccccccc} u_{11}x_1 & + & u_{12}x_2 & + & \dots & + & u_{1n}x_n & = & c_1 \\ & & u_{22}x_2 & + & \dots & + & u_{2n}x_n & = & c_2 \\ & & & & \ddots & & \vdots & & \vdots \\ & & & & & & u_{nn}x_n & = & c_n \end{array}$$

durch Rückwärtssubstitution

$$x_n = c_n/u_{nn}, \quad x_i = \left(c_i - \sum_{j=i+1}^{n} u_{ij}x_j\right)/u_{ii} \quad (i \leq n-1).$$

Quiz

Was ist die algorithmische Komplexität der Rückwärtssubstitution?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Warum Matrixzerlegungen?

Grundidee: “Teile und herrsche” für Matrizen

Analogie: Primfaktorzerlegung von Zahlen

- $84 = 2^2 \cdot 3 \cdot 7$
- Vereinfacht Rechnung: $84 \cdot 126 = (2^2 \cdot 3 \cdot 7) \cdot (2 \cdot 3^2 \cdot 7) = 2^3 \cdot 3^3 \cdot 7^2$

Für Matrizen: $\boldsymbol{A} = \boldsymbol{BC}$ mit “einfachen” $\boldsymbol{B}, \boldsymbol{C}$

- **Dreiecksmatrizen:** Lösen in $O(n^2)$ statt $O(n^3)$
- **Orthogonale Matrizen:** Numerisch stabil, einfach zu invertieren
- **Diagonalmatrizen:** Triviale Operationen

Vorteile von Zerlegungen:

1. Effiziente Lösung mehrerer LGS mit gleicher Matrix
2. Numerische Stabilität verbessern
3. Struktur und Eigenschaften von $\boldsymbol{A}$ erkennen
4. Grundlage für viele weitere Algorithmen (Spektralzerlegung & `SVD`, …)

Die LU-Zerlegung

Intuition: LU-Zerlegung

Idee: Die Gauss-Elimination zerlegt $\boldsymbol{A}$ in:

- $\boldsymbol{L}$ (**L**ower): speichert alle Eliminationsschritte
- $\boldsymbol{U}$ (**U**pper): die resultierende obere Dreiecksmatrix

Vorteil: Sobald wir $\boldsymbol{A} = \boldsymbol{LU}$ einmal berechnet haben, können wir $\boldsymbol{Ax} = \boldsymbol{b}_i$ für *jedes* $\boldsymbol{b}_i$ effizient lösen:

Ax = b $\Longrightarrow$ **LUx = b** $\Longrightarrow$

1. Löse **Ly = b**
2. Löse **Ux = y**

Die LU-Zerlegung

- Im ersten Schritt des Eliminationsverfahrens bringen wir $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ in Zeilenstufenform.
- Dazu gehen wir von links nach rechts durch die Spalten und entfernen alle Einträge unter der Diagonalen.
- Ist das aktuelle System $(\boldsymbol{M} \mid \boldsymbol{r})$, erreichen wir das für die k-te Spalte durch die Multiplikation einer Matrix $\boldsymbol{L}_k$ von links, wobei

$$\boldsymbol{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{pmatrix}$$

$$\text{mit } l_{ik} = m_{ik}/m_{kk}$$

Die LU-Zerlegung

- Insgesamt ist also $(\boldsymbol{U} \mid \boldsymbol{c})$ gegeben durch $\boldsymbol{U} = \boldsymbol{L}_{n-1} \cdots \boldsymbol{L}_1 \boldsymbol{A}$ und $\boldsymbol{c} = \boldsymbol{L}_{n-1} \cdots \boldsymbol{L}_1 \boldsymbol{b}$.
- Es gilt dann $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$, wobei $\boldsymbol{L} = \boldsymbol{L}_1^{-1} \cdots \boldsymbol{L}_{n-1}^{-1}$.

Die Inverse zu $\boldsymbol{L}_k$ macht deren Operation rückgängig (Vorzeichen kippen):

$$\boldsymbol{L}_k^{-1} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & l_{nk} & & 1 \end{pmatrix}$$

Jedes $\boldsymbol{L}_k^{-1}$ ist eine untere Dreiecksmatrix, also auch $\boldsymbol{L} = \boldsymbol{L}_1^{-1} \cdots \boldsymbol{L}_{n-1}^{-1}$ – genauer:

$$\boldsymbol{L} = \begin{pmatrix} 1 & & & & \\ l_{21} & \ddots & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & & \ddots & \ddots & \\ l_{n1} & \cdots & \cdots & l_{n(n-1)} & 1 \end{pmatrix}$$

$\Longrightarrow$ $\boldsymbol{L}$ sammelt einfach alle Eliminationskoeffizienten l_{ik} ein.

Beispiel: LU-Zerlegung für 3×3-Matrix

Sei $\boldsymbol{A} = \begin{pmatrix} 2 & 1 & -1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}$.

Farben: zu eliminieren, Pivot, Eliminationskoeffizienten

Schritt 1: Eliminiere Spalte 1 mit $\boldsymbol{L}_1$:

$$l_{21} = 4/2 = 2, \quad l_{31} = -2/2 = -1$$

$$\boldsymbol{L}_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -(-1) & 0 & 1 \end{pmatrix},$$

$$\boldsymbol{L}_1\boldsymbol{A} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 8 & 1 \end{pmatrix}$$

Schritt 2: Eliminiere Spalte 2 mit $\boldsymbol{L}_2$:

$$l_{32} = 8/(-8) = -1$$

$$\boldsymbol{L}_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -(-1) & 1 \end{pmatrix},$$

$$\boldsymbol{L}_2\boldsymbol{L}_1\boldsymbol{A} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} = \boldsymbol{U}$$

Ergebnis: $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ mit

$$\boldsymbol{L} = \boldsymbol{L}_1^{-1}\boldsymbol{L}_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad \boldsymbol{U} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix}$$

Die LU-Zerlegung zur Lösung von LGS

Theorem: LU-Zerlegung

Jede invertierbare Matrix $\boldsymbol{A}$ hat — ggf. erst nach Zeilenvertauschungen $\boldsymbol{A} \to \boldsymbol{PA}$ mit einer Permutationsmatrix $\boldsymbol{P}$ (→ *Pivotierung*, s.u.) — eine Darstellung

$$\boldsymbol{A} = \boldsymbol{LU} \qquad \text{bzw.} \qquad \boldsymbol{PA} = \boldsymbol{LU}$$

mit

- einer unteren Dreiecksmatrix $\boldsymbol{L}$,
- einer oberen Dreiecksmatrix $\boldsymbol{U}$.

- Eine Zerlegung ohne Pivotierung (ohne Zeilenvertauschungen, also mit $\boldsymbol{P} = \boldsymbol{I}$) erhalten wir genau dann, wenn bei der Gauß-Elimination kein Pivotelement null wird. Wird ein Pivot null, müssen wir vorher Zeilen vertauschen.
- Da $\boldsymbol{L}$ und $\boldsymbol{U}$ Dreiecksmatrizen sind, können wir das LGS $\boldsymbol{Ax} = \boldsymbol{b}$ bzw. $\boldsymbol{LUx} = \boldsymbol{b}$ jetzt leicht lösen:
  1. Löse $\boldsymbol{Ly} = \boldsymbol{b}$ (bzw. $\boldsymbol{Ly} = \boldsymbol{Pb}$) durch Vorwärtssubstitution.
  2. Löse $\boldsymbol{Ux} = \boldsymbol{y}$ durch Rückwärtssubstitution.
- **Beachte:** Die Zerlegung $\boldsymbol{A} = \boldsymbol{LU}$ ist für alle $\boldsymbol{b}$ gleich. Wir müssen sie also nur einmal ausführen.

Beispiel: Lösen mit LU-Zerlegung (Fortsetzung)

Mit $\boldsymbol{L}, \boldsymbol{U}$ von vorher und $\boldsymbol{b} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix}$.

Schritt 1: Vorwärtssubstitution für $\boldsymbol{Ly} = \boldsymbol{b}$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix} \quad \Longrightarrow \quad \begin{aligned} y_1 &= 5 \\ y_2 &= -2 - 2 \cdot 5 = -12 \\ y_3 &= 9 - (-1) \cdot 5 - (-1) \cdot (-12) = 9 + 5 - 12 = 2 \end{aligned}$$

Schritt 2: Rückwärtssubstitution für $\boldsymbol{Ux} = \boldsymbol{y}$:

$$\begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -12 \\ 2 \end{pmatrix} \Longrightarrow \begin{aligned} x_3 &= 2/3 \\ x_2 &= (-12 - 2 \cdot (2/3))/(-8) = 5/3 \\ x_1 &= (5 - 1 \cdot (5/3) - (-1) \cdot (2/3))/2 = 2 \end{aligned}$$

Pivotierte LU-Zerlegung

- Die Methode scheitert, falls im k-ten Schritt $m_{kk} = 0$.
- Das lässt sich vermeiden, indem wir Zeilen von $\boldsymbol{A}$ tauschen: $\rightarrow$ **Pivotierung**
  - löst $\boldsymbol{PA} = \boldsymbol{LU}$ mit Permutationsmatrix $\boldsymbol{P}$
- *pivoting* verbessert Stabilität auch schon für $m_{kk} \approx 0$.

Quiz

Im k-ten Schritt zum Finden von $\boldsymbol{L}$ und $\boldsymbol{U}$ multiplizieren wir

$$\boldsymbol{M} = \begin{pmatrix} * & \cdots & * & \cdots & * \\ & \ddots & \vdots & \ddots & \vdots \\ & & * & \cdots & * \\ & & \vdots & \ddots & \vdots \\ & & * & \cdots & * \end{pmatrix} \text{ mit } \boldsymbol{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{pmatrix}.$$

Was ist die Komplexität dieses Schritts?

1. $O(n^2)$
2. $O(n^3)$
3. $O((n-k)^2)$
4. $O(k^3)$

Komplexität

- Die Berechnung der LU-Zerlegung hat also die Komplexität

$$\sum_{k=1}^{n-1} O((n-k)^2) = \sum_{k=1}^{n-1} O(n^2) = O(n^3).$$

- Das müssen wir nur einmal ausführen und können damit mehrere LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}_j$, $j = 1, \ldots, J$, lösen.
- Für Vorwärts- und Rückwärtssubstitution brauchen wir je $O(n^2)$.
- Insgesamt ist die Laufzeit also $O(n^3 + Jn^2)$.

- Die Laufzeit kann verbessert werden, wenn $\boldsymbol{A}$ besondere Eigenschaften erfüllt, z.B.:
  - symmetrisch: $\boldsymbol{A} = \boldsymbol{A}^\top$,
  - positiv definit: $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} > 0$ für alle $\boldsymbol{x} \neq \boldsymbol{0}$,
  - Bandeigenschaft: $a_{ij} = 0$ für alle $|i - j| > \beta$,
  - dünn besetzt (engl.: *sparse*): $a_{ij} = 0$ für die allermeisten (i, j).
- Für all das gibt es hervorragende Implementierungen. Man schreibt diese Algorithmen selten selbst.

Cholesky-Zerlegung

SPD-Matrizen

Ein Spezialfall ist in der Statistik besonders wichtig:

SPD-Matrizen sind

- **s**ymmetrisch und
- **p**ositiv **d**efinit.

Def.: SPD-Matrix

Eine Matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ heißt **SPD**, wenn

- $\boldsymbol{A} = \boldsymbol{A}^\top$ (*symmetrisch*) und
- $\boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x} > 0 \; \forall \; \boldsymbol{x} \in \mathbb{R}^n \setminus \{\boldsymbol{0}\}$ (*positiv-definit*).

- Symmetrisches $\boldsymbol{A}$ ist **positiv-*semi*definit** (*PSD*) $\iff \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x} \geq 0 \; \forall \; \boldsymbol{x} \in \mathbb{R}^n$

SPD- und PSD-Matrizen in Statistik und ML

1. **Kovarianzmatrizen**: $\boldsymbol{\Sigma} = \mathbb{E}[(\boldsymbol{x} - \boldsymbol{\mu})(\boldsymbol{x} - \boldsymbol{\mu})^\top]$
   sind immer PSD, und SPD, wenn $\boldsymbol{x}$ nicht auf einer Untermenge von $\mathbb{R}^p$ mit Dimension $< p$ liegt.
   - Multivariate Normalverteilungen $\sim \mathcal{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, Hauptkomponentenanalyse (PCA)
2. **Gram-Matrizen**: $\boldsymbol{K} = \boldsymbol{X}^\top \boldsymbol{X}$
   - Normalgleichungen $\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{\beta} = \boldsymbol{X}^\top \boldsymbol{y}$ für *Kleinste-Quadrate*-Probleme; Kernel-Matrizen (SVMs, Gauss-Prozesse, …)
3. **Hesse-Matrizen**: $\boldsymbol{H} = \left[\frac{\partial^2}{\partial x_i \partial x_j} f(\boldsymbol{x})\right]_{i,j}$
   - Fisher-Information; *Konvexität* & *Newton-Verfahren* in der Optimierung

Warum so wichtig?

SPD-Matrizen garantieren:

- Existenz und — bei positiver Diagonale von $\boldsymbol{L}$ — Eindeutigkeit der Cholesky-Zerlegung
- Numerische Stabilität der Cholesky-Zerlegung (auch ohne Pivotierung)
- Alle Eigenwerte sind (reell und) positiv $\implies$ voller Rang

Cholesky-Zerlegung

- Für SPD-Matrizen kann man $\boldsymbol{L}$ und $\boldsymbol{U}$ so wählen, dass $\boldsymbol{U} = \boldsymbol{L}^\top$.

Theorem: Cholesky-Zerlegung

Jede SPD-Matrix $\boldsymbol{A}$ kann in $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^\top$ zerlegt werden, wobei $\boldsymbol{L}$ eine untere Dreiecksmatrix mit positiver Diagonale ($l_{ii} > 0$) ist. Diese Zerlegung ist eindeutig.

- Der Rechen- und Speicheraufwand ist hier etwa halb so groß wie für die volle LU-Zerlegung.

(Beweis per Induktion → Anhang)

Beispiel: Cholesky-Zerlegung

Berechne Cholesky-Zerlegung von $\boldsymbol{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$.

Ansatz: $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^\top$ mit $\boldsymbol{L} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$

Ausmultiplizieren:

$$\boldsymbol{L}\boldsymbol{L}^\top = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Koeffizientenvergleich:

$$\begin{aligned} l_{11}^2 = 4 \quad &\Longrightarrow \quad l_{11} = 2 \\ l_{11}l_{21} = 2 \quad &\Longrightarrow \quad l_{21} = 2/2 = 1 \\ l_{21}^2 + l_{22}^2 = 3 \quad &\Longrightarrow \quad l_{22} = \sqrt{3-1} = \sqrt{2} \end{aligned}$$

Ergebnis:

$$\boldsymbol{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix} \Longrightarrow \qquad \boldsymbol{L}\boldsymbol{L}^\top = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Cholesky-Zerlegung

Beispielanwendung:

- Sei $\boldsymbol{x} \in \mathbb{R}^d$ ein Zufallsvektor mit $\mathbb{E}[\boldsymbol{x}] = \mathbf{0}$ und $\mathbb{V}\mathrm{ar}(\boldsymbol{x}) = \boldsymbol{I}_d = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$.
- Sei $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ eine SPD Matrix und $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^\top$.
- Dann hat $\boldsymbol{y} = \boldsymbol{L}\boldsymbol{x}$ die Kovarianz

$$\mathbb{V}\mathrm{ar}(\boldsymbol{y}) = \mathbb{E}\left[\boldsymbol{y}\boldsymbol{y}^\top\right] = \mathbb{E}\left[\boldsymbol{L}\boldsymbol{x}\boldsymbol{x}^\top\boldsymbol{L}^\top\right] = \boldsymbol{L}\mathbb{E}\left[\boldsymbol{x}\boldsymbol{x}^\top\right]\boldsymbol{L}^\top = \boldsymbol{L}\boldsymbol{L}^\top = \boldsymbol{\Sigma}.$$

(das mittlere Stück fällt weg: $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top] = \boldsymbol{I}_d$ nach Voraussetzung)

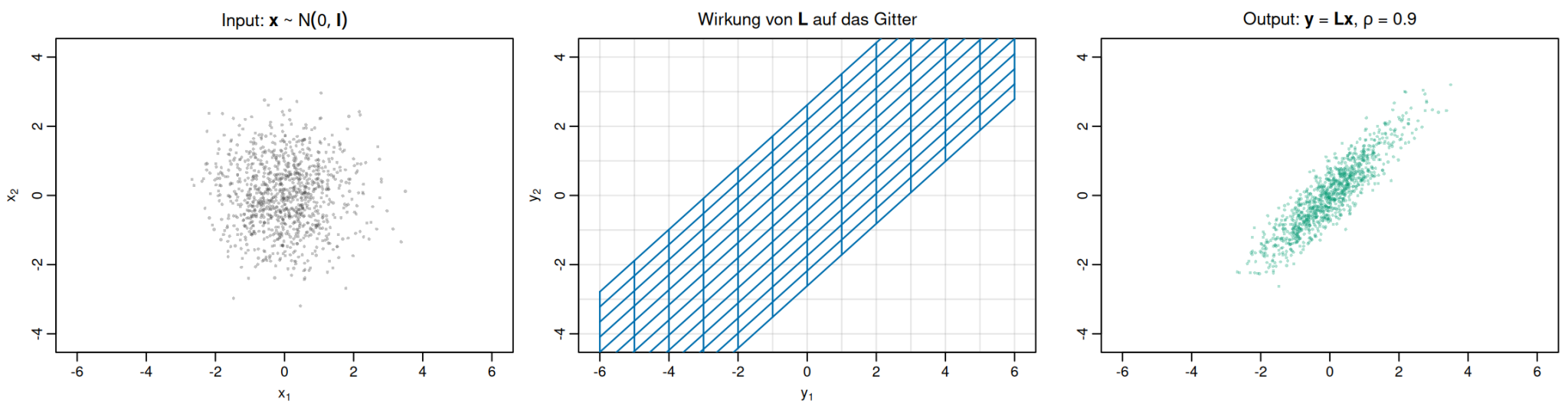
- Dies wird z.B. verwendet, um aus $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ zu simulieren:
  für $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_d)$ ist $\boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

Demo: Aus $\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$ simulieren

```r
set.seed(7)
Sigma <- matrix(c(1,   0.9,
                  0.9, 1), 2, 2)
# !! chol() liefert die obere Dreiecksmatrix R mit Sigma = R'R !!
L <- t(chol(Sigma))
x <- matrix(rnorm(2 * 1000), nrow = 2)
y <- L %*% x
round(cov(t(y)), 2)
```

```
     [,1] [,2]
[1,] 1.00 0.89
[2,] 0.89 0.99
```

$$\boldsymbol{L} = \begin{pmatrix} 1 & 0 \\ 0.9 & \sqrt{1 - 0.9^2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0.9 & 0.436 \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^\top.$$



$\Longrightarrow$ $\boldsymbol{L}$ verformt Gitter und Punktwolke auf dieselbe Weise; die empirische Kovarianz von $\boldsymbol{y}$ reproduziert $\boldsymbol{\Sigma}$.

Vergleich: LU und Cholesky

LU-Zerlegung

- **Ohne Pivotierung instabil** bei kleinen Pivotelementen
  - Division durch sehr kleine Zahlen führt zu numerischer Instabilität
- **Mit partieller Pivotierung** praktisch stabil (Higham, 2020)
- **Kondition:** Fehler $\approx \kappa(\boldsymbol{A}) \cdot \epsilon_{\text{Maschine}}$

Cholesky-Zerlegung

- **Natürlich stabil** ohne Pivotierung für *SPD-Matrizen*
- **Effizienz:** $\approx 2\times$ schneller als LU (halber Aufwand)
- **Robustheit:** In exakter Arithmetik bricht das Verfahren für **jede** nicht positiv definite Matrix ab (Wurzel aus einer nicht-positiven Zahl); bei echten SPD-Matrizen passiert das nur durch Rundungsfehler und nur bei **extrem** schlechter Kondition
  - Der Abbruch ist ein nützlicher *Diagnosehinweis*: $\boldsymbol{A}$ ist nicht SPD oder numerisch grenzwertig
- **Pivotierte Variante:** Anwendung auch für *semi*-positiv-definite Matrizen! (Details $\rightarrow$ Anhang)

Fazit

- *LU*: Universell einsetzbar, benötigt Pivotierung für Stabilität
- *Cholesky*: optimal für *SPD-Matrizen* (Statistik!), numerisch überlegen

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- **Invertiere niemals eine Matrix!**
- *LU*- und *Cholesky*-Zerlegungen

Nächste Vorlesung

- Singulärwertzerlegung (*SVD*)

Self-Check

Frage 1: Sie müssen viele LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}_j$ mit derselben SPD-Matrix $\boldsymbol{A}$ lösen. Bestes Vorgehen?

1. Für jedes $\boldsymbol{b}_j$ die Gauss-Elimination komplett neu durchführen.
2. Einmal $\boldsymbol{A}^{-1}$ berechnen, dann jeweils $\boldsymbol{A}^{-1}\boldsymbol{b}_j$ multiplizieren.
3. Einmal Cholesky $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^\top$, dann je zweimal Substitution.
4. Einmal pivotierte LU, da sie für SPD-Matrizen stabiler als Cholesky ist.

Frage 2: Wann braucht die LU-Zerlegung Pivotierung?

1. Nur wenn $\boldsymbol{A}$ singulär ist.
2. Wenn ein Pivotelement m_{kk} null oder nahe null ist.
3. Nie — sie verbessert nur die Laufzeit.
4. Nur für symmetrische Matrizen.

Frage 3: $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ liegt bereits vor. Kosten, um *ein* LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ zu lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. $O(n^2 + n^3)$

Anhang

Cholesky-Zerlegung (Induktionsbeweis)

- $n = 1$: Weil $A \in \mathbb{R}$ SPD ist, gilt $A > 0$. Dann ist $A = LL^\top$ mit $L = \sqrt{A}$.
- $n > 1$:
  - Sei $\boldsymbol{A} = \begin{pmatrix} a & \boldsymbol{c}^\top \\ \boldsymbol{c} & \boldsymbol{B} \end{pmatrix}$ und die Hypothese gelte für $n-1$.
  - Setze $\boldsymbol{l} = \frac{\boldsymbol{c}}{\sqrt{a}}$. ($\boldsymbol{A}$ SPD $\implies a > 0$, da $a = \boldsymbol{e}_1^\top \boldsymbol{A} \boldsymbol{e}_1$.)
  - Außerdem ist für jedes $\boldsymbol{x} \in \mathbb{R}^{n-1} \setminus \{\boldsymbol{0}\}$:

$$\boldsymbol{x}^\top (\boldsymbol{B} - \boldsymbol{l}\boldsymbol{l}^\top)\boldsymbol{x} = \begin{pmatrix} -\frac{\boldsymbol{c}^\top \boldsymbol{x}}{a} \\ \boldsymbol{x} \end{pmatrix}^\top \boldsymbol{A} \begin{pmatrix} -\frac{\boldsymbol{c}^\top \boldsymbol{x}}{a} \\ \boldsymbol{x} \end{pmatrix} > 0$$

    also ist $\left(\boldsymbol{B} - \boldsymbol{l}\boldsymbol{l}^\top\right)$ SPD und $\left(\boldsymbol{B} - \boldsymbol{l}\boldsymbol{l}^\top\right) = \widetilde{\boldsymbol{L}}\widetilde{\boldsymbol{L}}^\top$ (Induktionshypothese).
  - Wähle nun

$$\boldsymbol{L} = \begin{pmatrix} \sqrt{a} & \boldsymbol{0}^\top \\ \boldsymbol{l} & \widetilde{\boldsymbol{L}} \end{pmatrix} \implies \boldsymbol{L}\boldsymbol{L}^\top = \begin{pmatrix} a & \boldsymbol{c}^\top \\ \boldsymbol{c} & \boldsymbol{l}\boldsymbol{l}^\top + \widetilde{\boldsymbol{L}}\widetilde{\boldsymbol{L}}^\top \end{pmatrix} = \boldsymbol{A} \quad \checkmark$$

  - *Eindeutigkeit:* $l_{11} = \sqrt{a}$ und $\boldsymbol{l} = \boldsymbol{c}/\sqrt{a}$ sind erzwungen, $\widetilde{\boldsymbol{L}}$ ist eindeutig nach Induktionsannahme.

↩ zurück: „Cholesky-Zerlegung"

Cholesky mit Pivotierung

- Notwendig falls $\boldsymbol{A}$ singulär oder numerisch rangdefizient
  - für positiv **semi**-definite Matrizen ($\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \geq 0$ statt > 0):
- **Complete pivoting:** $\boldsymbol{P}^\top \boldsymbol{A}\boldsymbol{P} = \boldsymbol{L}\boldsymbol{L}^\top$
  - Permutationsmatrix $\boldsymbol{P}$ (zeilen- und spaltenvertauschend)
  - **Rang-aufdeckend:** Nullspalten in $\boldsymbol{L}$ zeigen Rangabfall von $\boldsymbol{A}$
- **Stabilität:** Auch für semidefinite Matrizen stabil (Higham, 1990)
- **Anwendungen** in ML/Statistik:
  - Kernel-Matrizen (Gauss-Prozesse, SVMs, …)
  - Niedrig-rangige approximative Matrixzerlegung: $O(k^2 n)$ statt $O(n^3)$
  - Gram-Matrizen $\boldsymbol{X}^\top \boldsymbol{X}$ bei $p > n$ (mehr Features als Daten)

↩ zurück: „Vergleich: LU und Cholesky"